Outlook

Lateral tech challenge

From Raluca Lăcătuș <raluca.lacatus@lateralgroup.com>
Date Tue 03/11/2026 11:02
To philip.hayward@outlook.com <philip.hayward@outlook.com>

Hi Phil,

Thanks again for our chat earlier. As mentioned, I'm sending you our tech challenge, if you could send your solution by the beginning of next week that would be great!

Scenario: *A CMS sends live updates via a webhook/event pipeline.*
Events include, but are not limited to:
- Add → new entities are being created.
- Update → an existing entity was successfully changed.
- Delete → an entity was removed or unpublished. Please note that unpublishing the entity does not remove it from the CMS, but disables it.

Once an entity is updated (new fields, new values etc), a new version is set for the entity in the CMS. New data is only available upon publishing the event! This will send an event containing the latest entity version. An "unpublish" event will contain the entity fields, as well as the version that is being unpublished.

Please keep in mind that an entity with version X can be modified → resulting in version X+1. Then, this gets unpublished. However, since there was no published version before that, you do no longer have the latest version in your database. Please also treat this corner case appropriately!

The CMS also handles confidential data, and must not be served publicly. Since the system does not provide any information about which data is of the said tier, you have to assume all data is restricted. The entities should not be publicly available. Please authenticate your endpoints so that only allowed users can access the REST API. You can use the same basic auth used for the CMS, but the users should be different. One is an organization request (the CMS sending updates), the other is user requests.

Task: Build a .NET 9 service that ingests those events, processes them, stores them in a database, and exposes them via a clean API.

Requirements:

1. **Data ingestion layer**
   - Expose a webhook endpoint: /cms/events
   - Secure your endpoint using Basic Authentication.
     - Username: feel free to use any string with a length of 10 to 20 characters.
     - Password: random guid.
   - Accept batch events. The first version added is version 1, and you can get multiple revisions per entity. Schema example below:

```
[
{ "type": "publish", "id": "X", "payload": {...}, "version": 2, "timestamp":
```

```
”2024-01-01T00:00:00Z”},
{ "type": "delete", "id": "Y", "timestamp”: ”2024-01-01T00:00:00Z” },
{ "type": "unPublish", "id": "Z", "payload": {...}, "version": 4, "timestamp
”: ”2024-01-01T00:00:00Z”},
]
```

2. Application Layer
- Incoming data must be validated and sanitized.
- Deleted entities should be removed (hard-delete), while unpublish should still keep the data in your persistence layer.

3. Data storage
- EF Core + Relational database of your choice.
- Entities must keep track of the latest data version.

4. REST API
- Provide endpoints that support listing entities for consumers.
  - If the user is an admin (feel free to decide which userId/email corresponds to that of an admin), it will receive all entities that a normal user can see, including the ones that were disabled. No need to implement separate endpoints, feel free to decide how to approach this!
- Data can not be updated by any kind of users, but an admin can disable them from the API - this will not affect the CMS, it’s an overwrite that does not affect CMS data!

5. Performance
- Feel free to use any asynchronous mechanisms to process incoming requests. This can also be synchronous, but we would like to understand why either decision was taken.
- Use a read-only/writer configuration for your application context and optimize your EF read queries.

6. Observability
- Log processed events, including failing ones.

7. Testing
- Test how events are processed and whether the ingestion constraints are successfully followed by the service.
- Test your basic authentication mechanism with valid/invalid user-password combinations.

8. Deliverable
- .NET Core 9 or above solution.
- Provide a README with setup/running instructions.
- The solution should work on either Mac or Windows, so build it platform-agnostic.
- Github repo with all the above.

***Notes to consider**: This assignment is intentionally open-ended. Focus on the correctness, structure, and clarity of the system. Correct event processing is the most important part. Feel free to make use of available LLMs to help you achieve the goal proposed in this assignment!*

Looking forward to seeing what you build!
--

LATERAL

Raluca Lacatus
Senior Tech Recruiter

Connect on LinkedIn
+40 770253508
lateralgroup.com